Министерство образования и науки Республики Башкортостан

Государственное автономное профессиональное образовательное учреждение

Уфимский колледж статистики, информатики и вычислительной техники

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____ З.З. Курмашева
«_____»_____ 2026 г.

Курсовая работа

по дисциплине МДК 02.01 Технология разработки программного обеспечения

на тему: «Разработка мобильного приложения для прослушивания подкастов»

Специальность 09.02.07 «Информационные системы и программирование»

Руководитель курсовой работы
_____О.В. Фатхулова
«___» _____ 2026 г.

Студент гр. 23П-4
_____ И.Ф. Файзельгаянов
«___» _____ 2026 г.

Уфа – 2026

Данная курсовая работа посвящена проектированию и разработке мобильного приложения для прослушивания подкастов с использованием современных кроссплатформенных технологий и асинхронного серверного REST API. В работе проанализированы основные принципы построения мобильных медиаплееров и систем дистрибуции потокового аудиоконтента. Описаны архитектурные преимущества и технические особенности выбранных программных средств.

Первая глава представляет характеристики и теоретические аспекты проектирования клиент-серверной информационной системы как средства структурирования и доставки медиаданных. Также эта глава объясняет обоснованность выбора реляционной СУБД MySQL в качестве хранилища данных, фреймворка FastAPI в качестве бэкенда и Flutter в качестве мобильной платформы с приведением подробных характеристик проектируемой базы данных.

Вторая глава описывает практическую разработку и программное внедрение серверного REST API в инфраструктуру приложения. Далее раскрывается эффективность реализации механизмов JWT-авторизации, валидации входящих .mp3 аудио файлов по размеру и MIME-типу, а также проектирование адаптивного графического интерфейса мобильного клиента.

Рассмотрены принципы работы асинхронных сетевых запросов и управления состоянием приложения через паттерн Provider, а также проведен приемочный анализ работоспособности системы. В заключении работы предложены рекомендации по дальнейшему масштабированию хранилища данных и оптимизации битрейта аудиофайлов в зависимости от пропускной способности сети.

В заключении представлены основные выводы и практические результаты по проделанной работе.

Основными источниками информации для написания работы послужили труды отечественных и зарубежных авторов в области проектирования баз данных, разработки мобильных систем и веб-служб, а также официальная

техническая документация фреймворков FastAPI и Flutter.

Тема курсовой работы: «Разработка мобильного приложения для прослушивания подкастов»

Ключевые слова: мобильное приложение, подкасты, Flutter, FastAPI, СУБД MySQL, REST API, управление состоянием, JWT-авторизация, потоковое аудио.

Объект исследования – процессы публикации, категоризации и воспроизведения аудиоконтента на мобильных устройствах.

Цель – проектирование и программная реализация клиент-серверного мобильного приложения для прослушивания и администрирования подкастов с применением кроссплатформенных программных средств.

В работе предложен анализ архитектурных решений для мобильного стриминга звука, рассмотрены способы физического хранения аудио файлов на сервере, показаны механизмы разграничения прав доступа по ролям (Слушатель и Администратор).

Результат работы является показателем актуальности внедрения легковесных асинхронных API для раздачи медиаданных без использования дорогостоящих сторонних облачных платформ.

В курсовой работе на тему: «Разработка мобильного приложения для прослушивания подкастов» описаны программные компоненты серверной логики на Python и архитектура мобильного клиента во Flutter, приведены методы защиты пользовательских сессий посредством шифрования токенов JWT.

Выбраны оптимальные средства и пакеты для обеспечения бесперебойного аудиовещания в режиме реального времени на мобильных устройствах.

Курсовая работа посвящена разработке и внедрению информационной системы дистрибуции подкастов. В ней описаны сведения о структуре реляционных таблиц, о принципе работы асинхронного сервера, показана установка зависимостей, настройка сетевого взаимодействия между эмулятором Android и локальным хостом, а также развертывание мобильного клиента. Использование JWT-авторизации и хэширования паролей по алгоритму bcrypt

обеспечивает сохранность персональных данных пользователей и предотвращает несанкционированный доступ к административным функциям управления контентом.

Человеческий фактор при вводе данных, попытки загрузки вредоносных или слишком объемных файлов — все это потенциальные риски стабильности системы, которые были устранены на этапе проектирования. Именно встроенная серверная валидация FastAPI и клиентские ограничения Flutter гарантируют полное отсутствие критических сбоев при некорректных действиях пользователей.

Кроссплатформенная разработка на базе Flutter и FastAPI — это эффективное решение для проектирования современных ориентированных на пользователя мобильных приложений.

Объектом исследования являются процессы публикации, категоризации и воспроизведения аудиоконтента на мобильных устройствах.

В курсовую работу входит содержание, введение, две главы (теоретическая и практическая), заключение и список использованных источников.

В введении раскрывается актуальность темы, ставятся цель и задачи исследования.

В аналитической части рассматриваются понятие клиент-серверной архитектуры, особенности потокового вещания аудио данных и осуществляется проектирование структуры базы данных СУБД MySQL.

В практической части подробно рассматриваются программные инструменты реализации REST API на FastAPI и построение графического интерфейса мобильного клиента с плеером на Flutter.

Заключение содержит выводы по проделанной работе и достигнутым результатам.

Задание к выполнению получил «30» марта 2026г.

Студент И.Ф. Файзельгаянов

Срок окончания «27» июня 2026г.

Руководитель О.В. Фатхулова

Задание рассмотрено на заседании цикловой комиссии информатики

«25» марта 2026 г. протокол № 5

Председатель цикловой комиссии О.В. Фатхулова

СОДЕРЖАНИЕ

лист

ВВЕДЕНИЕ	7
1. Общая часть	9
1.1. Анализ предметной области и обоснование актуальности разработки	9
1.2. Обоснование выбора архитектуры и технологического стека	10
1.3. Спецификация и структура реляционной базы данных	11
2. Специальная часть	13
2.1. Реализация серверного приложения	13
2.2. Разработка пользовательского интерфейса и логики	16
2.2.1. Архитектура клиентских модулей	18
2.3. Протокол тестирования разработанной информационной системы	19
3. Приложение А. Контрольный пример	24
4. Приложение Б. Исходный код программы	26
4.1. Код сервера	26
4.2. Код клиентского мобильного приложения	33
5. Приложение В. Результат работы	36
ЗАКЛЮЧЕНИЕ	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	44

ВВЕДЕНИЕ

Развитие современных беспроводных сетей передачи данных и повсеместное распространение смартфонов привели к качественному изменению способов потребления медиаконтента. Одним из наиболее динамично растущих сегментов цифрового вещания стали аудиоподкасты — регулярные разговорные или образовательные передачи, транслируемые через интернет. Специфика мобильного использования требует от подобных систем высокой оптимизации сетевого трафика, плавного воспроизведения потокового аудио в реальном времени и интуитивно понятного интерфейса, адаптированного для работы «на ходу».

Разработка проприетарного программного решения для прослушивания подкастов позволяет избежать ограничений крупных публичных платформ, таких как отсутствие гибкой категоризации, жесткие рамки монетизации и зависимость от внешних облачных инфраструктур. Использование клиент-серверной архитектуры с выделенным реляционным хранилищем и кроссплатформенным мобильным клиентом обеспечивает необходимый уровень масштабируемости, отказоустойчивости и автономии.

Объектом исследования является процесс публикации, структурирования и дистрибуции потокового аудиоконтента (подкастов) посредством мобильных систем.

Предметом исследования является проектирование и программная реализация клиент-серверного мобильного приложения для прослушивания подкастов на базе СУБД MySQL, серверного REST API на FastAPI и кроссплатформенного клиента на Flutter.

Целью курсовой является создание масштабируемой и производительной информационной системы, обеспечивающей надежную авторизацию пользователей, удобную навигацию и контекстный поиск по аудиокаталогу, а также бесперебойное воспроизведение подкастов на мобильных устройствах.

Для достижения цели работы были определены следующие задачи:

- изучить теоретические аспекты проектирования клиент-серверных архитектур мобильных приложений и потокового вещания медиаданных;
- разработать реляционную модель базы данных СУБД MySQL для учета учетных записей и аудиофайлов;
- спроектировать и разработать серверную часть информационной системы (REST API) на базе асинхронного веб-фреймворка FastAPI;
- реализовать клиентскую часть мобильного приложения на фреймворке Flutter с использованием паттерна управления состоянием Provider;
- провести функциональное тестирование разработанной системы для подтверждения стабильности её работы при граничных нагрузках и вводе некорректных данных.

1. Общая часть

1.1. Анализ предметной области и обоснование актуальности разработки

Современные системы хранения и дистрибуции подкастов представляют собой сложные программные комплексы, работающие по модели «клиент-сервер». Основная задача таких систем — предоставить пользователю быстрый доступ к аудиофайлам без необходимости их предварительного скачивания целиком на устройство, что реализуется с помощью технологии прогрессивного скачивания (progressive download) или потокового вещания (streaming).

С точки зрения контент-менеджмента, предметная область характеризируется наличием двух основных ролей:

- слушатель (Client/Listener): регистрируется в системе, осуществляет поиск подкастов по ключевым словам, фильтрует каталог по категориям и прослушивает аудиозаписи с помощью интерактивного плеера;
- администратор (Admin): осуществляет глобальное администрирование системы, включая просмотр пользователей, добавление новых подкастов (с метаданными и аудиофайлами) и удаление устаревшего контента.

Актуальность разработки собственного приложения обусловлена необходимостью локального хранения медиаресурсов на собственных мощностях предприятия, гибкого управления базой пользователей и исключения рисков несанкционированного доступа к служебным или образовательным аудиоматериалам организации.

1.2. Обоснование выбора архитектуры и технологического стека

Для реализации проекта была выбрана классическая трехзвенная клиент-серверная архитектура, включающая в себя уровень представления (мобильный клиент), уровень бизнес-логики (REST API сервер) и уровень хранения данных (СУБД).

В качестве серверного фреймворка выбран FastAPI. FastAPI является одним из наиболее производительных веб-фреймворков на Python благодаря интеграции библиотек Starlette и Pydantic. Использование стандарта асинхронного программирования ASGI (uvicorn) позволяет серверу эффективно обрабатывать тысячи параллельных запросов ввода-вывода (I/O-bound), что критически важно при раздаче аудиофайлов. Для взаимодействия с СУБД используется SQLAlchemy ORM, автоматизирующая построение безопасных SQL-запросов.

Для разработки мобильного интерфейса выбран кроссплатформенный фреймворк Flutter. Использование компилируемого языка Dart обеспечивает высокую производительность графики за счет собственного движка отрисовки Impeller. Flutter позволяет использовать единую кодовую базу для сборки под платформы Android и iOS. Управление состоянием (State Management) реализовано через пакет Provider, разделяющий визуальную верстку и логику работы с данными.

Для хранения структурированной информации выбрана реляционная СУБД MySQL. Она обеспечивает высокую скорость чтения транзакций, надежность хранения данных и полную совместимость со стандартом SQL.

1.3. Спецификация и структура реляционной базы данных

Схема базы данных спроектирована с учетом обеспечения целостности и минимизации избыточности данных. Ниже представлены таблицы 1.3.1 – 1.3.2 физической структуры базы данных MySQL.

Таблица 1.3.1 – Структура таблицы users (учетные записи пользователей)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор пользователя	id	PRIMARY KEY	первичный, авто-инкремент
Электронная почта пользователя	email	VARCHAR(150)	уникальное, не пустое
Хэш-код пароля	hashed_password	VARCHAR(255)	не пустое
Роль доступа	role	VARCHAR(50)	стандартное значение listener

Таблица 1.3.2 – Структура таблицы podcasts (каталог подкастов)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор подкаста	id	PRIMARY KEY	первичный, авто-инкремент
Название аудиозаписи	title	VARCHAR(255)	не пустое
Категория подкаста	category	VARCHAR(100)	не пустое
Описание содержания подкаста	description	VARCHAR(1000)	не пустое
Путь к аудиозаписи на сервере	file_path	VARCHAR(500)	не пустое
Дата и время публикации	created_at	DATETIME	стандартное, текущее время с датой

2. Специальная часть

2.1. Реализация серверного приложения

Серверная часть функционирует как REST API, обмениваясь с мобильным клиентом данными в формате JSON. Безопасность эндпоинтов реализована на базе авторизации по протоколу OAuth2 с использованием Bearer JWT-токенов подписи HS256.

Для хэширования паролей на стороне бэкенда используется библиотека bcrypt, исключающая хранение паролей пользователей в чистом виде.

Часть разработанных API-эндпоинтов серверной части представлена на рисунках 2.1.1 – 2.1.2:

```python
@router.post("/register", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
def register(user_data: UserCreate, db: Session = Depends(get_db)):
    # Проверяем, существует ли уже пользователь с таким email.
    db_user = db.query(User).filter(User.email == user_data.email).first()
    if db_user:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Пользователь с таким email уже зарегистрирован"
        )
      You, 3 часа назад • working
    # Создаем нового пользователя.
    hashed_pw = hash_password(user_data.password)
    new_user = User(
        email=user_data.email,
        hashed_password=hashed_pw,
        role="listener"  # По умолчанию регистрируем с ролью Слушателя.
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

@router.post("/login", response_model=Token)
def login(user_data: UserLogin, db: Session = Depends(get_db)):
    # Поиск пользователя.
    db_user = db.query(User).filter(User.email == user_data.email).first()
    if not db_user or not verify_password(user_data.password, db_user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Неверный email или пароль"
        )

    # Генерация JWT.
    access_token = create_access_token(data={"sub": db_user.email, "role": db_user.role})
    return Token(access_token=access_token, role=db_user.role)

@router.get("/users", response_model=List[UserResponse])
def list_users(
    db: Session = Depends(get_db),
    current_admin: User = Depends(get_current_admin)
):
    """Возвращает список всех зарегистрированных пользователей (только для Администратора)."""
    return db.query(User).all()
```

Рисунок 2.1.1 – Часть разработанных API-эндпоинтов серверной части

```python
# Публикация подкаста (только для роли `Администратора`).      You, 2 часа назад • comple
@router.post("/", response_model=PodcastResponse, status_code=status.HTTP_201_CREATED)
async def create_podcast(
    title: str = Form(...),
    category: str = Form(...),
    description: str = Form(None),
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_admin: User = Depends(get_current_admin)
):
    # Проверка расширения файла.
    if not file.filename.lower().endswith(".mp3"):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Разрешены только файлы формата .mp3"
        )

    # Проверка MIME-типа.
    if file.content_type != "audio/mpeg":
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Неверный MIME-тип. Разрешен только audio/mpeg"
        )

    # Валидация размера файла (до 100 МБ).
    if file.size is not None:
        file_size = file.size
    else:
        # Резервный способ через прямое обращение к внутреннему файловому объекту.
        file.file.seek(0, 2)
        file_size = file.file.tell()
        file.file.seek(0)

    if file_size > 100 * 1024 * 1024:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Размер файла превышает лимит в 100 МБ"
        )
```

Рисунок 2.1.2 – Часть разработанных API-эндпоинтов серверной части

2.2. Разработка пользовательского интерфейса и логики

Интерфейс мобильного приложения разработан в соответствии с гайдлайнами Material Design 3 и ориентирован на темную цветовую палитру (Dark Premium) для снижения нагрузки на зрение при длительном использовании.

Проектирование функциональных возможностей мобильного приложения базируется на принципе разделения прав доступа между двумя категориями пользователей: Слушателем и Администратором. Для формализации требований к системе и описания сценариев взаимодействия пользователей с интерфейсом была разработана диаграмма прецедентов (Use Case Diagram).

Данная диаграмма позволяет на концептуальном уровне визуализировать иерархию функций приложения, выделив ключевые действия, доступные каждому из акторов. Сценарии использования включают в себя как базовые операции по работе с контентом (просмотр каталога, воспроизведение подкастов), так и административные функции по управлению базой данных и публикацией новых медиафайлов.

На рисунке 2.2.1 представлена Use Case диаграмма.

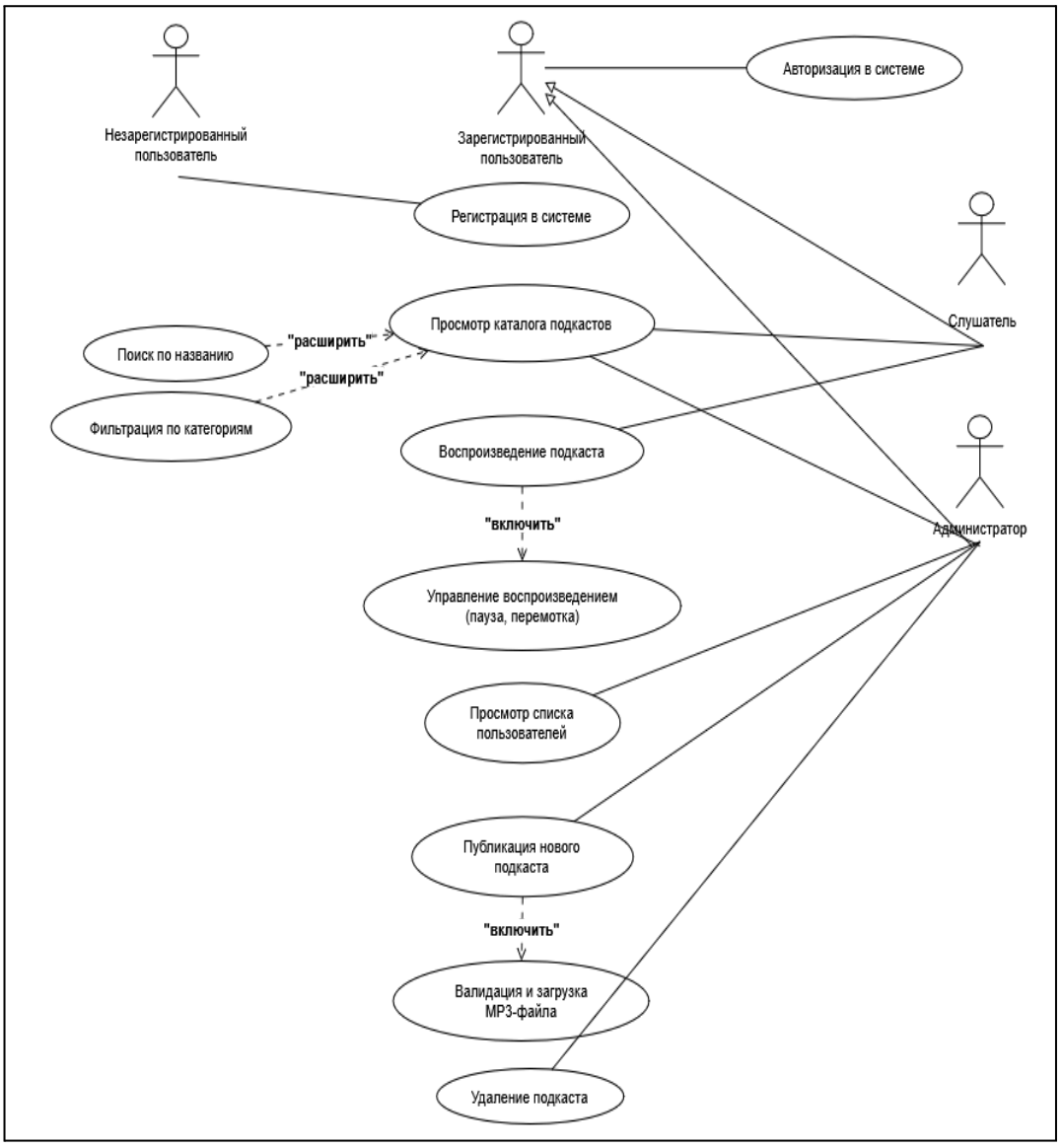


Рисунок 2.2.1 – Use Case диаграмма

2.2.1. Архитектура клиентских модулей

Логика приложения разделена на три ключевых провайдера:

- модуль AuthProvider сохраняет JWT-токен во внутреннем хранилище устройства через shared_preferences и управляет сессией пользователя;
- модуль PodcastProvider запрашивает список подкастов у API сервера, обрабатывает контекстный поиск и горизонтальную фильтрацию по чипсам категорий;
- модуль PlaybackProvider инкапсулирует работу с нативным аудиоплеером смартфона посредством пакета audioplayers и отслеживает текущую позицию вещания, буферизацию, общую длительность трека и статус воспроизведения.

Интерфейс плеера (BottomSheet) - для экрана воспроизведения реализована интерактивная выдвижная панель. Чтобы избежать конфликтов переполнения разметки на этапе изменения размеров панели в момент анимации раскрытия (от 84px до 275px), используется адаптивный контейнер SingleChildScrollView.

2.3. Протокол тестирования разработанной информационной системы

В ходе приемочного тестирования системы были проверены все критические сценарии использования программы.

В таблицах 2.3.1 – 2.3.4 представлены протоколы тестирования.

Таблица 2.3.1 – Протокол тестирования успешной регистрации слушателя

Параметр	Описание
Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование регистрации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при регистрации нового слушателя
Этапы теста	Открыть форму регистрации; заполнить поля «Email», «Пароль», «Повторный пароль»; нажать кнопку «Зарегистрироваться»
Тестовые данные	Email: testingList@gmail.com, Пароль: 7878488Aa, Повторный пароль: 7878488Aa
Ожидаемый результат	Учётная запись создана
Фактический результат	Учётная запись создана. Результат представлен на рисунке 2.3.1



Рисунок 2.3.1 – Фактический результат

Таблица 2.3.2 – Протокол тестирования регистрации слушателя с уже занятой почтой

Параметр	Описание
1	2
Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование регистрации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при попытки регистрации нового слушателя с уже существующей почтой
Этапы теста	Открыть форму регистрации; заполнить поля «Email (уже существующего слушателя)», «Пароль», «Повторный пароль»; нажать кнопку «Зарегистрироваться»

Продолжение таблицы 2.3.2

1	2
Тестовые данные	Email: testingList@gmail.com, Пароль: 7878488Aa, Повторный пароль: 7878488Aa
Ожидаемый результат	Ошибка, учетная запись не создана
Фактический результат	Ошибка, учетная запись не создана. Результат представлен на рисунке 2.3.2



Рисунок 2.3.2 – Фактический результат

Таблица 2.3.3 – Протокол тестирования успешной аутентификации

Параметр	Описание
1	2
Дата теста	25.06.2026
Приоритет тестирования	Высокий

Продолжение таблицы 2.3.3

1	2
Заголовок/название теста	Тестирование аутентификации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при аутентификации пользователя
Этапы теста	Открыть форму авторизации; заполнить поля «Email», «Пароль»; нажать кнопку «Авторизоваться»
Тестовые данные	Email: testingList@gmail.com, Пароль: 7878488Aa
Ожидаемый результат	Успешная аутентификация, получение JWT-токена
Фактический результат	Успешная аутентификация, получение JWT-токена. Результат представлен на рисунке 2.3.3

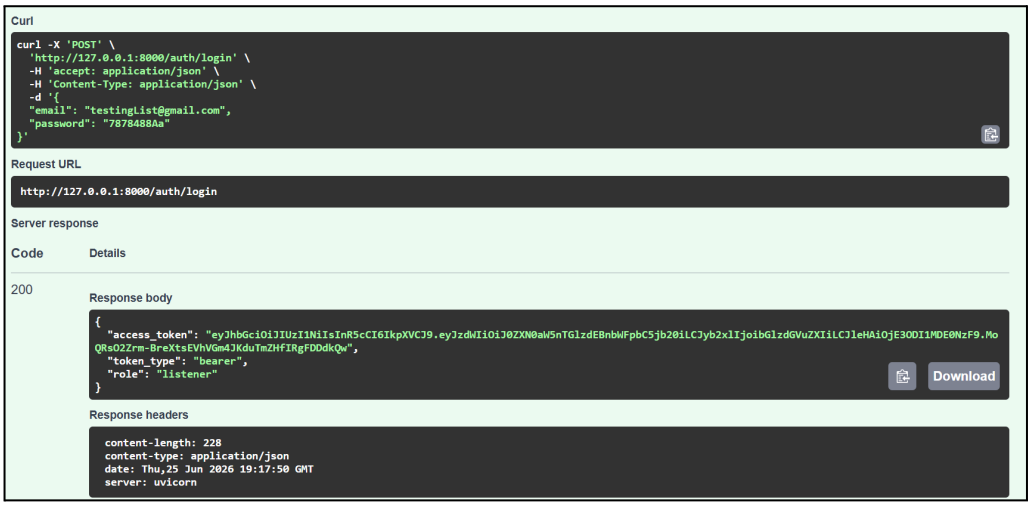


Рисунок 2.3.3 – Фактический результат

Таблица 2.3.4 – Протокол тестирования аутентификации с неверными данными

Параметр	Описание
Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование аутентификации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при попытки аутентификации пользователя с некорректными данными
Этапы теста	Открыть форму авторизации; заполнить поля «Email», «Пароль»; нажать кнопку «Авторизоваться»
Тестовые данные	Email: aa@gsd, Пароль: -
Ожидаемый результат	Ошибка, неудачная аутентификация
Фактический результат	Ошибка, неудачная аутентификация. Результат представлен на рисунке 2.3.4

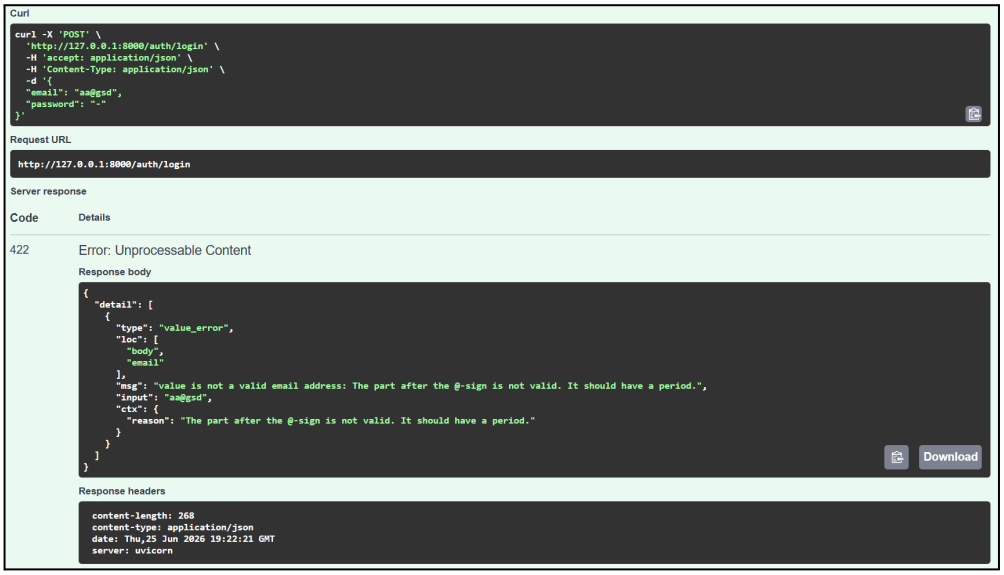


Рисунок 2.3.4 – Фактический результат

3. Приложение А. Контрольный пример

Таблица 3.1 – Входные данные для таблицы users (учетные записи пользователей)

Электронная почта	Пароль	Роль
aorayden@gmail.com	*<i51V7CEkgS	Admin
listener@gmail.com	*<i51V7CEkgS	Listener

Таблица 3.2 – Входные данные для таблицы podcasts (подкасты)

Название	Категория	Описание	Путь к файлу	Дата и время создания
test	Искусство и культура	test	/static/21447b95d51149fea6a3323524213d20.mp3	2026-06-25 20:47:14

Таблица 3.3 – Выходные данные при формировании нового подкаста

Название	Категория	Описание	Путь к файлу	Дата и время создания
Арктика	Искусство и культура	Арктика	Автоматическая генерация	2026-06-26 09:03:58

4. Приложение Б. Исходный код программы

Полный исходный код проекта содержит более 20 файлов и доступен в полном объеме на электронном носителе, приложенном к курсовой работе. В настоящем приложении представлены ключевые файлы системы.

4.1. Код сервера

main.py:

```
import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from .database import engine, Base
from .routers import auth, podcasts

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Podcast App API", version="1.0.0")

# Автоматически создаем папку для сохранения файлов, если её нет.
os.makedirs("uploads", exist_ok=True)

# Монтируем раздачу статических файлов по адресу /static.
app.mount("/static", StaticFiles(directory="uploads"), name="static")

# Подключаем роутеры.
app.include_router(auth.router)
```

```
app.include_router(podcasts.router)


@app.get("/")
def read_root():
    return {"status": "ok", "message": "Podcast API is working"}


config.py:
import os

# Секретный ключ для подписи токенов JWT. Нужно будет потом убрать.
SECRET_KEY = os.getenv("SECRET_KEY", "your_super_secret_jwt_key_here_change_me_in_production")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24  # Срок действия токена - 24 часа.


database.py:
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "mysql+pymysql://root:*<i51V7CEkgS@localhost:3306/podcast_db"


engine = create_engine(
    DATABASE_URL,
```

```
        pool_pre_ping=True  # Проверка стабильности соединения перед запросом.
    )
    SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
    Base = declarative_base()

    # Генератор для получения сессии БД в end-points.
    def get_db():
        db = SessionLocal()
        try:
            yield db
        finally:
            db.close()
```

dependencies.py:

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.orm import Session
import jwt
from .database import get_db
from .models import User
from .config import SECRET_KEY, ALGORITHM
```

```
    # HTTPBearer автоматически генерирует поле ввода Bearer-токена в Swagger UI
    security_scheme = HTTPBearer()

    def get_current_user(
        credentials: HTTPAuthorizationCredentials = Depends(security_scheme),
        db: Session = Depends(get_db)
    ) -> User:
            """Проверка валидности токена и возвращение текущего пользователя."""
        token = credentials.credentials
        try:
                payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            email: str = payload.get("sub")
            if email is None:
                raise HTTPException(
                    status_code=status.HTTP_401_UNAUTHORIZED,
                    detail="Недействительный токен авторизации",
                    headers={"WWW-Authenticate": "Bearer"},
                )
        except jwt.PyJWTError:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Недействительный токен авторизации",
```

```
            headers={"WWW-Authenticate": "Bearer"},
        )

    user = db.query(User).filter(User.email == email).first()
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Пользователь не найден"
        )
    return user


def get_current_admin(current_user: User = Depends(get_current_user)) -> User:
        """Дополнительная проверка: является ли пользователь администратором."""
    if current_user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Доступ запрещен. Требуются права администратора."
        )
    return current_user
```

models.py:

```
from sqlalchemy import Column, Integer, String, DateTime, func
from .database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(150), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
     role = Column(String(50), default="listener", nullable=False)  # 'listener' или 'admin'.

class Podcast(Base):
    __tablename__ = "podcasts"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(255), nullable=False)
    category = Column(String(100), nullable=False)
    description = Column(String(1000), nullable=True)
    file_path = Column(String(500), nullable=False)  # Путь к MP3 файлу на сервере.
    created_at = Column(DateTime, server_default=func.now())
```

security.py:

```
import bcrypt
import jwt
from datetime import datetime, timedelta, timezone
from .config import SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES


def hash_password(password: str) -> str:
    """Хэширование пароля с солью."""
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt)
    return hashed.decode('utf-8')


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Сравнение чистого пароля с сохраненным хэшем."""
    try:
        return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))
    except Exception:
        return False


def create_access_token(data: dict) -> str:
    """Генерация JWT-токена доступа."""
    to_encode = data.copy()
```

```
        expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
        encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

4.2. Код клиентского мобильного приложения

main.dart:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'providers/auth_provider.dart';
import 'providers/playback_provider.dart';
import 'providers/podcast_provider.dart';
import 'screens/admin_screen.dart';
import 'screens/login_screen.dart';
import 'screens/main_screen.dart';

void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        ChangeNotifierProvider(create: (_) => PodcastProvider()),
        ChangeNotifierProvider(create: (_) => PlaybackProvider()),
```

```
      ],
child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final authProvider = context.watch<AuthProvider>();
    return MaterialApp(
      title: 'Podcasts App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.dark,
        scaffoldBackgroundColor: const Color(0xFF121216),
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF7C4DFF),
          brightness: Brightness.dark,
          primary: const Color(0xFF9E77FA),
          secondary: const Color(0xFF00E676),
          surface: const Color(0xFF1A1A22),
        ),
```

```
appBarTheme: const AppBarTheme(
  backgroundColor: Color(0xFF121216),
  elevation: 0,
  centerTitle: true,
  titleTextStyle: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
    color: Colors.white,
  ),
),
elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: const Color(0xFF7C4DFF),
    foregroundColor: Colors.white,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    textStyle: const TextStyle(
      fontWeight: FontWeight.bold,
      fontSize: 16,
    ),
  ),
),
chipTheme: ChipThemeData(
```

```
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
          ),
          side: BorderSide.none,
        ),
      ),
      home: _getHomeScreen(authProvider),
    );
  }

  Widget _getHomeScreen(AuthProvider authProvider) {
    if (!authProvider.isAuthenticated) {
      return const LoginScreen();
    }
    if (authProvider.role == 'admin') {
      return const AdminScreen();
    }
    return const MainScreen();
  }
}
```

5. Приложение В. Результат работы

На рисунках 5.1 – 5.6 представлен результат курсовой работы.

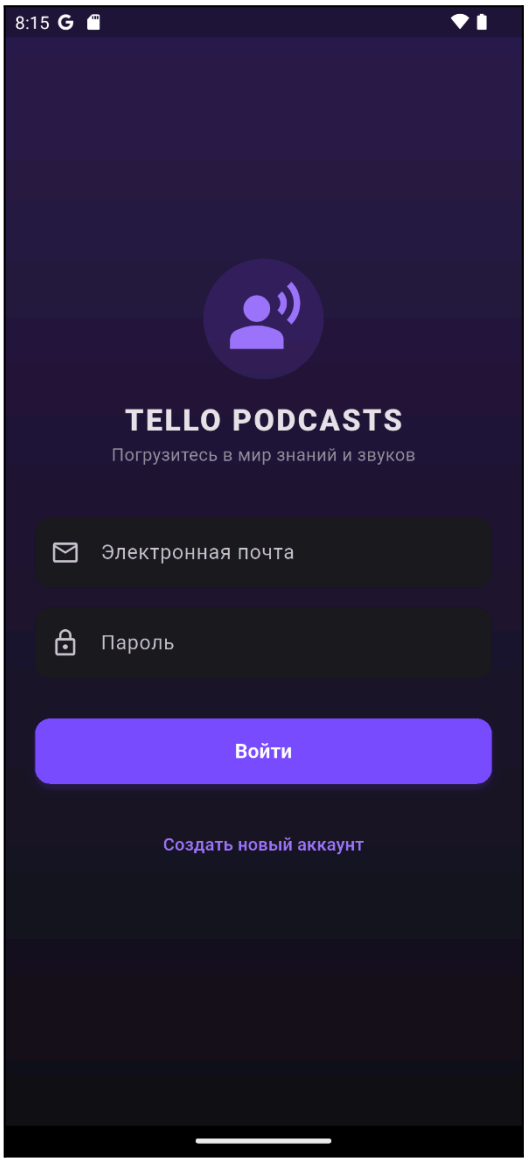


Рисунок 5.1 – Экран авторизации

					40.Ф-830-2026 09.02.07 КП-ПЗ		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Файзельгаянов И.Ф.			Разработка мобильного приложения для прослушивания подкастов	Лит.	Лист 2 / Листов 43
Провер.		Фатхулова О.В.					
						УКСИВТ 23П-4	

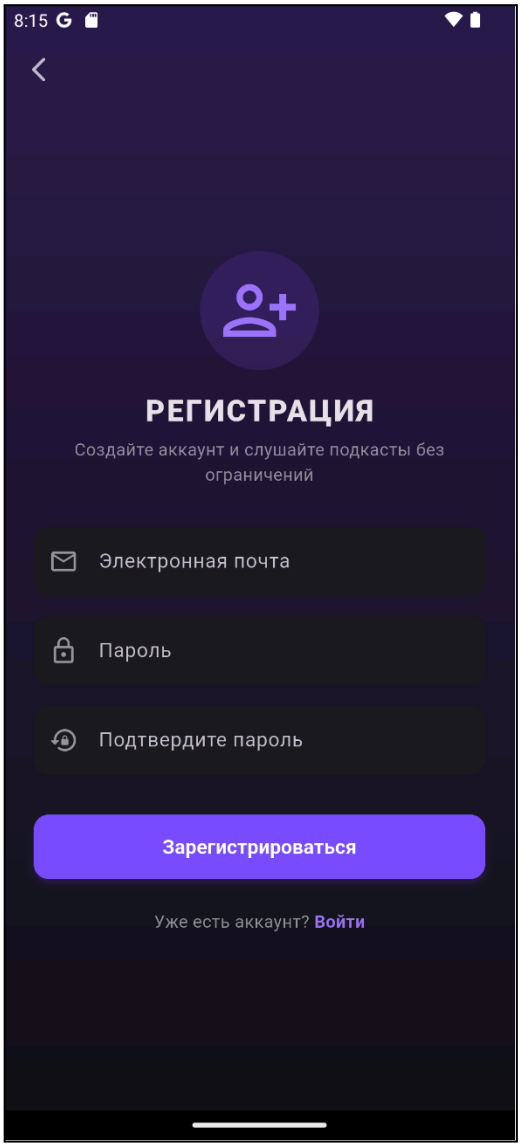


Рисунок 5.2 – Экран регистрации

					40.Ф-830-2026 09.02.07 КП-ПЗ		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Файзельгаянов И.Ф.			Разработка мобильного приложения для прослушивания подкастов	Лит.	Лист 2 / Листов 43
Провер.		Фатхулова О.В.					
						УКСИВТ 23П-4	

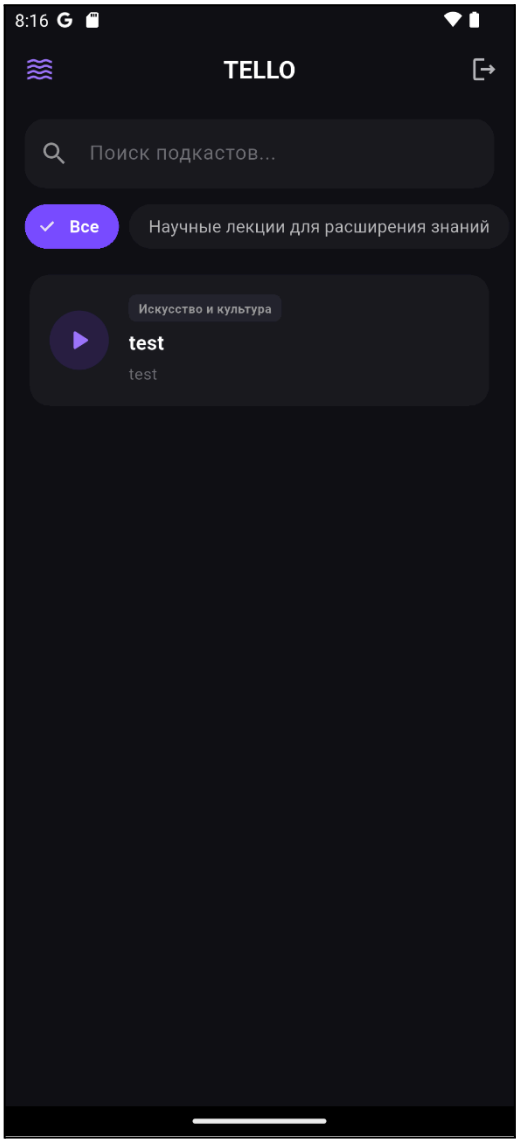


Рисунок 5.3 – Экран пользователя.

					40.Ф-830-2026 09.02.07 КП-ПЗ		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Файзельгаянов И.Ф.			Разработка мобильного приложения для прослушивания подкастов	Лит.	Лист 2 / Листов 43
Провер.		Фатхулова О.В.					
						УКСИВТ 23П-4	



Рисунок 5.4 – Экран администратора, вкладка Подкасты.

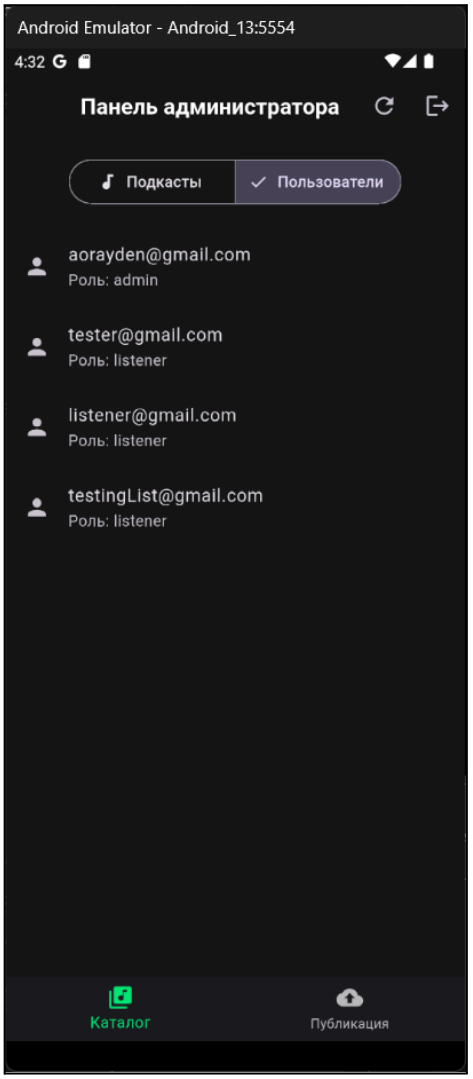


Рисунок 5.5 – Экран администратора, вкладка Пользователи.

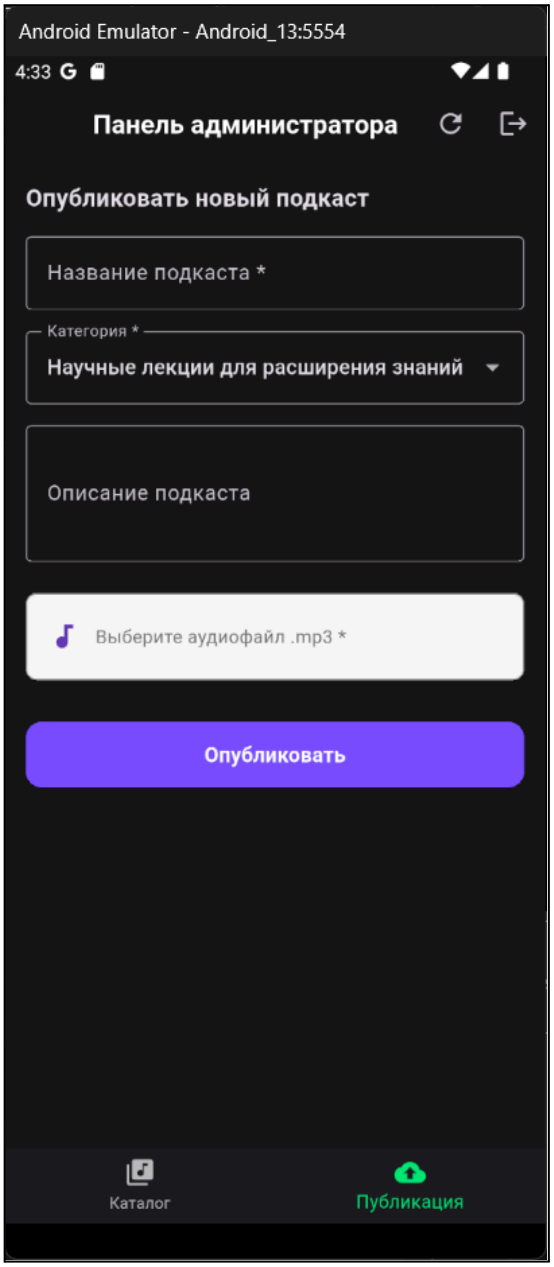


Рисунок 5.6 – Экран администратора, страница Публикация.

					40.Ф-830-2026 09.02.07 КП-ПЗ	
Изм.	Лист	№ докум.	Подпись	Дата		
Разраб.		Файзельгаянов И.Ф.			Разработка мобильного приложения для прослушивания подкастов	Лит. / Лист 2 / Листов 43
Провер.		Фатхулова О.В.				
						УКСИВТ 23П-4

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была полностью спроектирована и реализована клиент-серверная информационная система для публикации и прослушивания подкастов.

В процессе выполнения проекта были достигнуты следующие результаты:

- разработана реляционная физическая схема базы данных на СУБД MySQL, обеспечивающая хранение данных учетных записей пользователей и атрибутов подкастов без избыточности;
- создано серверное REST API на базе FastAPI (Python) с асинхронным циклом обработки запросов (uvicorn), безопасным односторонним хэшированием паролей алгоритмом bcrypt и JWT-авторизацией, а также реализованы строгие серверные правила проверки размера и расширения загружаемых аудиоматериалов;
- разработано кроссплатформенное мобильное приложение на базе фреймворка Flutter (Dart) с использованием паттерна управления состоянием Provider;
- спроектирован современный и адаптивный пользовательский интерфейс в темном стиле, удобный для длительного взаимодействия.

Все задачи курсовой работы решены в полном объеме, цель работы успешно достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Издания

1. «Современный стиль программирования» (2021, Питер) – Автор Б. Любанович.
2. «Flutter в действии» (2020, Москва) – Автор Х. Кингсли.
3. «Базы данных: учебник» (2021, Москва) – Автор Кумскова И.А.

Электронные ресурсы:

1. fastapi.tianglo.com
2. docs.flutter.dev